

Maverick: Leveraging Address Translation to Mitigate Memory Latency in Server Workloads

Ho Je Lee*, Minchan Kim*, Jaewon Kwon*, Jain Koo*, Yongju Lee*, Won Hur*[†], Jiwon Lee[‡], and Won Woo Ro*

* Yonsei University, Seoul, Republic of Korea

[†] LG Electronics, Seoul, Republic of Korea

[‡] Uppsala University, Uppsala, Sweden

Email: {hoje.lee, minchan.kim, jaewon.kwon, jain.koo, yongju.lee, won.hur, wro}@yonsei.ac.kr, jiwon.lee@it.uu.se

Abstract—Long memory latency remains a major performance bottleneck in modern processors. To mitigate this cost, processors rely on the cache hierarchy, which is designed for low-latency access on a hit. On a miss, however, the request pays the lookup latency of each cache that misses before reaching a lower level in the memory hierarchy. This paper identifies two highly exploitable characteristics of address translation that can be used to eliminate part of the on-chip cache latency. First, when a translation misses in the TLB hierarchy, the resulting page walk fetches translation data such as page-table entries through the cache hierarchy. Because dedicated structures such as TLBs and page walk caches filter out high-locality translation accesses before cache access, page walk requests exhibit fundamentally different locality characteristics from instruction/data requests. Second, because address translation precedes dependent instruction and data accesses, the level at which a translation is resolved in the translation hierarchy correlates with the level at which the resulting instruction and data requests are ultimately served in the memory hierarchy. A similar trend also holds within the translation process itself, where later page walk requests follow the locality implied by the earlier translation requests.

Based on these observations, we propose *Maverick*, a mechanism that exploits address translation to reduce on-chip cache-traversal latency for translations as well as instruction and data accesses, all of which are critical bottlenecks in server workloads. *Maverick* bypasses ineffective upper-level cache accesses for translation requests and uses translation-driven off-chip prediction to overlap long-latency memory accesses with cache traversal. Our evaluation with 359 Google server traces from the 4th Data Prefetching Championship shows that *Maverick* reduces stall cycles due to off-chip memory requests by 20.99% on average and improves geomean performance by 5.12%, outperforming prior off-chip predictors.

Index Terms—Address Translation, Memory Hierarchy, Off-chip Prediction, Server Workloads

I. INTRODUCTION

Processors rely on cache hierarchies to reduce memory latency by keeping frequently reused data on chip. However, off-chip misses still pay the traversal latency of failed cache levels because requests typically probe L1, L2, LLC, and main memory sequentially to avoid the energy cost of parallel lookup. As a result, while the cache hierarchy is effective on hits, a miss pays not only the eventual lower-level access latency, but also the traversal latency of the levels that fail to serve it.

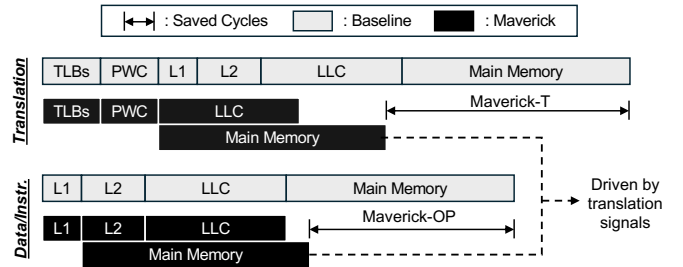


Fig. 1. Overview of *Maverick*, which leverages translation reuse behavior to accelerate both address translation and demand instruction/data accesses.

Off-chip prediction [3], [6], [9], [54] reduces this overhead by predicting whether a demand access will go off chip and issuing the memory request in parallel with cache traversal. Existing off-chip predictors, however, primarily target demand data accesses, leaving instruction fetches and address translation largely unaddressed. This limitation is particularly significant for modern server workloads, where large instruction and memory footprints cause significant stalls not only from data misses, but also from instruction fetches and translation misses [11], [14], [25], [26], [32], [44], [50].

We identify two key properties of address translation that create opportunities to reduce unnecessary cache-traversal latency across translation, instruction, and data accesses. First, translation requests that reach the cache hierarchy have low temporal locality, as TLBs and page walk caches already capture most frequently reused translations. Consequently, upper-level caches rarely serve page walk traffic, despite adding lookup latency and consuming cache capacity. Second, address translation lies at the head of a dependence chain that includes subsequent instruction/data accesses and later page-walk requests. This dependence creates a correlation between the translation outcome and where the subsequent accesses are served in the memory hierarchy. Translations resolved in upper levels tend to be followed by accesses also resolved in upper levels, whereas translations that proceed deeper are often followed by replayed instruction, data, and page-walk accesses handled deeper in the hierarchy. We also identify the *masking effect*, caused by the granularity mismatch between page-level translation and cache-line-level accesses. A page walk typically appears only on the first cache line access

to a page after a long reuse interval. Once the translation is installed in the TLB, later same-page accesses hit in the TLB even if their cache-line reuse distances remain long. We exploit this effect to extend translation-derived signals to later same-page accesses.

Based on these observations, we design *Maverick*, which uses address translation as a common signal to reduce on-chip traversal latency across translation, instruction, and data accesses. As shown in Fig. 1, *Maverick* has two components. *Maverick-T* targets translation by bypassing upper-level cache lookups and fills for low-locality traffic, forwarding translation requests directly to the LLC to shorten the critical path and reduce cache pollution. It also speculatively issues memory requests when leaf-level page walk accesses are likely to go off chip, overlapping main-memory access with the remaining translation path (Fig. 1, top). *Maverick-OP* similarly uses translation-derived signals to identify subsequent instruction fetches and data loads likely to go off chip, issuing their memory requests early to overlap main-memory access with cache traversal (Fig. 1, bottom).

We evaluate *Maverick* using diverse Google server traces from the 4th Data Prefetching Championship [4], which exhibit substantial long-latency stalls across data accesses, instruction fetches, and address translation. Across these workloads, *Maverick* consistently improves performance by reducing unnecessary cache-traversal latency across all three access types. In contrast to prior off-chip predictors that primarily target demand data accesses, *Maverick* exploits address translation both to accelerate page walks themselves and to identify subsequent instruction and data accesses that are likely to reach memory. As a result, *Maverick* outperforms state-of-the-art off-chip predictors [3], [9], [54] across these server workloads. These results demonstrate that address translation is not merely an additional source of memory overhead, but also a useful unified architectural signal for mitigating the diverse memory bottlenecks encountered in modern server processors.

II. BACKGROUND

A. Prefetching and Off-Chip Prediction

As memory and cache access latencies continue to grow, modern processors increasingly rely on latency-hiding techniques for performance. Existing approaches fall into two classes: (1) *prefetching* and (2) *off-chip prediction*. Prefetching brings cache blocks into the hierarchy before demand, converting future misses into hits when predictions are timely and accurate. In contrast, off-chip prediction operates when a demand access is issued. It predicts whether the access will miss in the caches and, if so, initiates the off-chip memory request early to overlap main memory access with cache traversal.

Prefetching has become specialized across access types because the signals that reveal future accesses differ for data, instruction, and translation streams. Data prefetchers are often PC-based, exploiting recurring stride or stream patterns generated by static load instructions [4], [5], [34], [37].

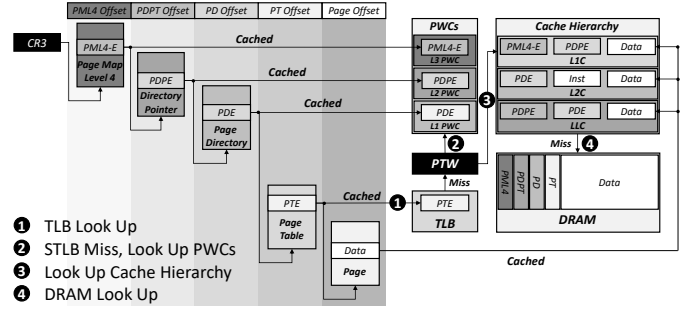


Fig. 2. Address translation process in x86-64 systems.

Instruction prefetchers instead rely on sequential or branch-guided mechanisms because instruction fetches are driven by control flow [16], [22], [35], [40], [41]. Translation prefetchers operate at page granularity and use page-level reuse signals to predict future TLB or page-table accesses [25], [50], [51]. This specialization reflects a broader principle: effective prediction must match the characteristics of each access stream. Off-chip prediction, however, has remained largely data-centric. Prior mechanisms focus predominantly on data loads [3], [6], [9], [54], using load history, PC-based signatures, or reuse-related load behavior to predict off-chip requests. These signals naturally fit data accesses but transfer less effectively to instruction fetches and address translation, whose behavior is shaped by control flow and page-level reuse.

B. Address Translation

In modern processors, every instruction fetch and data access must first be translated from a virtual address to a physical address before access. Modern systems perform virtual-to-physical translation using multi-level page tables [21], which can introduce significant latency due to pointer chasing. To mitigate this cost, processors employ hierarchical Translation Lookaside Buffers (TLBs), consisting of small, fast L1 TLBs backed by a larger shared second-level TLB (STLB), to cache recently used translations and avoid page table walks. In addition, Page Walk Caches (PWCs) [1], [7], [8] store intermediate page table entries (PTEs), reducing the number of memory accesses required during a page walk. With effective PWC coverage, the Page Table Walker (PTW) typically issues approximately 1–3 memory accesses per walk, depending on the workload. Fig. 2 illustrates the virtual-to-physical translation flow. For each translation request, the processor first probes the L1 TLB and then the STLB (1). If both TLBs miss, the PTW traverses the multi-level page table, reusing cached intermediate PTEs from the PWC when available (2) and issuing the remaining (mostly 1–3) dependent memory accesses. These accesses traverse the standard cache hierarchy, including L1, L2, and LLC (3), and proceed to main memory on a cache miss (4). Overall, the hierarchical translation process is designed to filter high-locality translation requests and minimize expensive full page table walks.

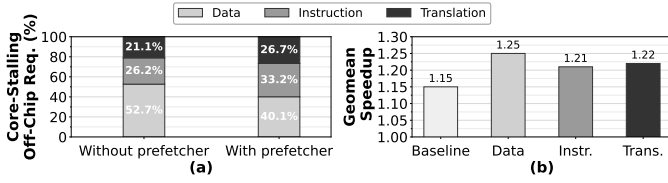


Fig. 3. (a) Core-stalling off-chip request breakdown without and with prefetching. (b) Geomean speedup with on-chip cache-hierarchy latency removed for each access class; *Baseline* denotes the prefetcher-enabled system, and all bars are normalized to the no-prefetcher configuration.

III. MOTIVATION

This section first shows that instruction, data, and translation accesses all become important sources of long-latency stalls in modern server workloads. We then identify two exploitable properties of address translation: poor upper-level locality after TLB/PWC filtering and strong correlation with dependent memory requests. Finally, we characterize the masking effect that extends this translation-derived correlation to later same-page accesses.

A. Impact of On-Chip Cache Latency

On-chip cache latency in off-chip requests is itself a major component of long-latency memory accesses. Before an off-chip access reaches main memory, it must first pay the lookup latency of L1, L2, and LLC. Prior studies on off-chip prediction report that, for off-chip loads, about 40.1% of the total latency is spent in on-chip cache [3], [6]. In modern server workloads, this bottleneck is not limited to data accesses alone, but also arises from instruction fetches and address translation. With modern prefetching enabled, core-stalling off-chip requests are no longer dominated by data accesses alone. To quantify this effect, we analyze Google server workloads from the 4th Data Prefetching Championship [4] under an Icelake-like configuration [13], using Berti [34] in L1D, Pythia [5] in L2, and FDIP [22] for instruction prefetching. As shown in Fig. 3(a), once these prefetchers are enabled, instruction and translation requests account for 33.2% and 26.7% of core-stalling off-chip requests, respectively, versus 40.1% for data. In other words, instruction, data, and translation accesses become much more comparable contributors to off-chip stall behavior after data-side misses are reduced. Moreover, instruction-side long-latency accesses remain difficult to fully eliminate with FDIP alone, because branch mispredictions and wrong-path fetches limit its ability to prefetch useful instructions early enough [16], [22].

Eliminating on-chip latency for instruction and translation accesses can have substantial performance impact, not just for data accesses. Fig. 3(b) quantifies this opportunity using an experiment in which on-chip cache latency is removed for off-chip requests of each access class. In our configuration, this corresponds to the serialized L1/L2/LLC traversal latency (20 cycles) before the request is sent to main memory. All bars report geomean speedup relative to the no-prefetcher configuration. The *Baseline* bar denotes the prefetcher-enabled system with Berti, Pythia, and FDIP, which already achieves

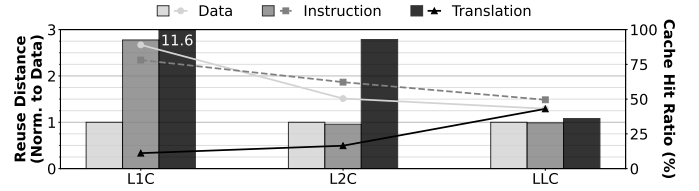


Fig. 4. Average reuse distance (number of interleaved accesses to the same cache set between reuse) and average cache hit ratio for access types across cache levels.

$1.15\times$ speedup. Even on top of this baseline, removing traversal latency raises performance to $1.25\times$, $1.21\times$, and $1.22\times$ for data, instruction, and translation accesses, respectively. These correspond to improvements of 8.7%, 5.2%, and 6.1% over the prefetcher-enabled baseline, showing that substantial traversal-latency headroom remains even after applying modern prefetchers.

B. Locality of Address Translation

Unlike instruction and data accesses, translation traffic reaches the cache hierarchy only after its high-locality portion has been filtered by TLBs and PWCs. Most high-locality translation entries are captured by these structures, and only when they miss does the page table walker issue pointer-chasing memory requests into the cache hierarchy [20], [39], [49]. Thus, the cache hierarchy observes only the low-locality tail of translation traffic, while instruction and data streams retain a broader locality mix at upper levels. To quantify this effect, we measure reuse distance and cache hit rate for instruction, data, and translation traffic at each cache level in an Icelake-like configuration [13]. Reuse distance, defined as the number of intervening accesses to the same cache set between two references to a block, serves as a proxy for temporal locality. As shown in Fig. 4, translation blocks have L1/L2 reuse distances approximately 2–12 $\times$ longer than instruction and data blocks, resulting in hit rates typically below 20%, compared to 50–90% for instruction and data.

At the LLC, translation no longer suffers this upper-level disadvantage because instruction and data accesses are also filtered before reaching the LLC. L1/L2 caches absorb many high-locality instruction and data references, so the remaining LLC-bound requests are mostly long-reuse accesses. Translation requests already enter the cache hierarchy as a low-locality tail after TLB/PWC filtering, making their LLC reuse behavior comparable to instruction and data. As a result, reuse distances and hit rates converge at the LLC. In other words, translation is an outlier in upper-level caches, but becomes competitive once instruction and data streams are similarly filtered. These observations indicate that translation is poorly suited for upper-level cache placement, but remains well matched to the LLC.

C. Replay Memory Requests

Prior work [9], [39] has shown that translation reuse distance is strongly correlated with the reuse distance of subsequent dependent data accesses. We build on this data-centric observation and examine whether the same translation-

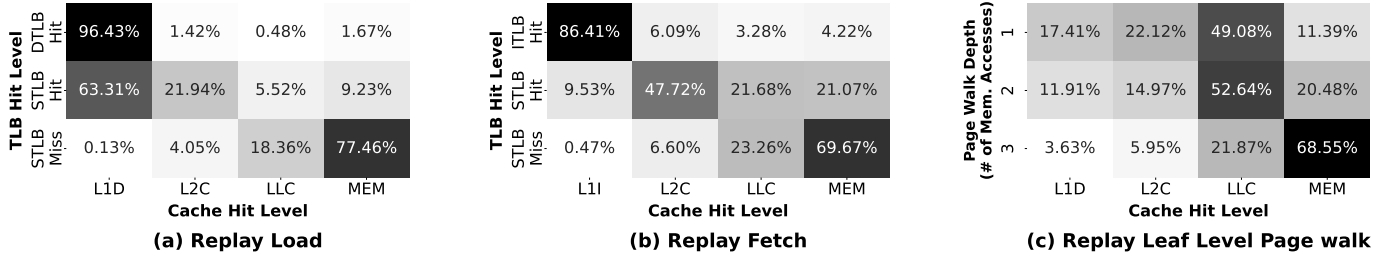


Fig. 5. Translation-dependent replay behavior. (a) and (b) show where replayed data loads and instruction fetches are served (L1/L2/LLC/Main memory) conditioned on the translation hit level (DTLB/ITLB, STLB, or STLB miss). (c) shows where the leaf-level page-walk access is served conditioned on page-walk depth (number of page walk memory requests); each row sums to 100%.

derived signal is useful beyond data loads. A single memory operation can expand into a dependence chain that includes translation-related page-table accesses and the replayed data load and instruction fetch once translation completes. Unlike data loads, for which existing predictors can exploit rich program-context features associated with the generating load instruction, replayed instruction fetches and translation requests expose fewer directly usable program-context features. Therefore, translation-derived signals provide an important alternative source of prediction information for replayed instruction fetches and translation requests. In the following, we characterize the replay effect for (i) *replayed loads/fetches*, defined as data loads and instruction fetches re-issued after translation completes, and (ii) *replayed leaf-level page-walk accesses*, defined as the final page-walk memory request that fetches the leaf PTE after any intermediate-level PTE accesses on PWC misses.

Replay Load/Fetch. The locality of a replayed instruction fetch or data load is strongly correlated with that of its preceding translation access. All instruction fetches and data accesses must be preceded by address translation, so when a translation exhibits a long reuse distance, the dependent instruction fetch or data load is also effectively revisited after a long interval. As a result, the replayed access tends to exhibit a similar reuse distance with the preceding translation. This coupling becomes stronger as the miss occurs deeper in the translation hierarchy: a DTLB/ITLB miss implies a longer translation reuse distance than a TLB hit, and an STLB miss implies an even longer translation reuse distance, which in turn stretches the effective reuse distance of the dependent replay.

Fig. 5(a),(b) illustrates this correlation by showing where replayed loads and instruction fetches are served, conditioned on the translation outcome. Each row corresponds to a translation hit level and sums to 100%, while columns indicate the cache level where the replayed access hits. When translation hits in the DTLB/ITLB, replayed accesses are largely served by upper-level caches, consistent with short reuse distances. When translation hits in the STLB, replayed accesses shift toward lower levels, reflecting longer reuse distances. Deep translation misses, especially STLB misses, are therefore a strong indicator that replayed accesses will go off chip. When translation misses in the STLB and triggers a page table walk, replayed loads and instruction fetches are predominantly

served by main memory, indicating that these replays themselves exhibit long reuse distances and rarely benefit from caches. This effect is reflected in Fig. 5(a),(b), where 77.46% of replayed loads and 69.67% of replayed instruction fetches reach main memory when the translation misses in the STLB.

Replay Leaf-Level Page Walk. A similar replay effect arises within the translation process itself. On an STLB miss, the page table walker issues a sequence of dependent memory requests across the page-table levels, ending with the leaf-PTE access, which we refer to as the *replayed leaf-level page-walk access*. If intermediate PTEs hit in the PWC, the walk requires only the leaf-PTE access. Otherwise, additional serialized pointer-chasing requests are needed. Similar to replayed loads and fetches, we find that misses at more intermediate PWC levels are followed by leaf-level accesses that are increasingly served by deeper levels of the memory hierarchy.

Fig. 5(c) quantifies this trend by grouping page walks according to the number of PTW memory requests. A *1-memory-access* walk fetches only the leaf PTE, while *2-memory-access* and *3-memory-access* walks additionally fetch one or two intermediate PTEs before the leaf. Walks requiring 4–5 accesses are extremely rare and are omitted for clarity. In the Google server workloads, additional Level 1 and Level 2 PTE fetches occur 0.85 and 0.52 times per walk on average, respectively. The fraction of leaf-level accesses reaching main memory increases from 11.39% for 1-memory-access walk to 20.48% and 68.55% for 2- and 3-memory-access walks, respectively. These results show that deeper page walks correlate with poorer leaf-PTE cache residency, making page-walk depth a useful indicator of off-chip translation traffic.

The 2-memory-access case reveals a more interesting behavior. Although its average main memory fraction is only 20.48%, this value hides substantial variation across traces: in some traces, the replayed leaf-level access rarely reaches main memory, whereas in others, it frequently does. We find that this difference arises from two distinct causes of the additional PWC miss. Because PWCs typically contain only a small number of entries, server workloads with large virtual-memory footprints can repeatedly evict intermediate PTEs from the PWC even when the corresponding leaf PTEs remain resident in the cache hierarchy. In this case, the additional PWC miss reflects PWC thrashing rather than poor leaf-PTE locality. In other cases, the PWC miss reflects genuinely long reuse

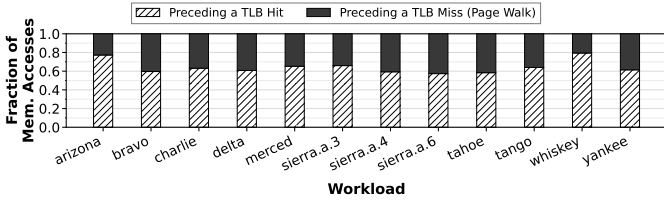


Fig. 6. Breakdown of demand main memory accesses into those occurring with a TLB hit (TLB-MEM) versus a page walk due to a TLB miss (PTW-MEM).

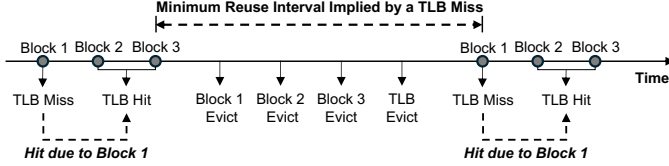


Fig. 7. Timeline of accesses to cache lines within the same page. The first access after a long interval incurs a TLB miss, whereas later accesses in the same page observe TLB hits.

distance, such that the intermediate PTE and the corresponding leaf PTE are both likely to have been evicted, causing the leaf-level access to reach main memory.

D. Page-Correlated Locality

While TLB miss events exhibit a strong correlation with long-latency demand accesses, a demand access that reaches main memory is not always preceded by a TLB miss. Fig. 6 breaks down all demand loads and instruction fetches that reach main memory into two cases: those whose translations hit in the TLB hierarchy (TLB-MEM) and those preceded by a page walk due to an STLB miss (PTW-MEM). On average, about 65% of main memory demand accesses occur on TLB hits (TLB-MEM), while the remaining 35% are directly preceded by an STLB-miss page walk (PTW-MEM). This implies that TLB miss events alone do not fully capture all long-latency demand behavior.

However, the strong correlation between translation reuse and replayed demand-access locality is not limited to TLB misses. It can also apply to some accesses that observe TLB hits. The key reason is the granularity mismatch between translation and caching. A TLB entry covers an entire virtual page, whereas demand accesses occur at cache-line granularity. Thus, when the first access to a page after a long interval installs the translation into the TLB, later first-touch accesses to other cache lines in the same page will observe TLB hits even though they may still be revisited only after similarly long intervals. Fig. 7 illustrates this masking effect using a timeline of accesses to cache blocks within the same page. After a sufficiently long interval with no accesses to the page, both the cached blocks and the page translation are evicted. When the page is revisited, the first access incurs a TLB miss. The distance from its prior access defines the minimum reuse interval implied by that miss at page granularity. This first access also reinstalls the page translation, causing subsequent

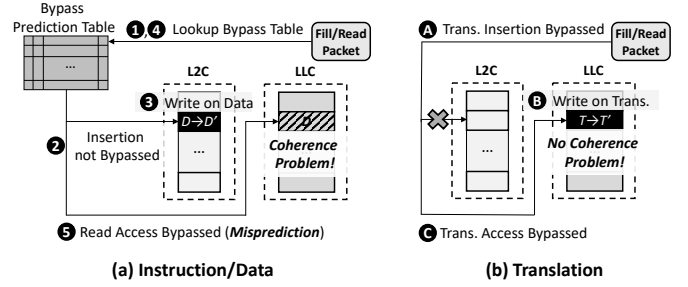


Fig. 8. (a) Bypassing accesses for instruction/data blocks risks coherence due to mispredictions, whereas (b) translation requests can bypass safely without prediction due to their uniformly low locality.

accesses to other cache blocks in the same page to observe TLB hits. These later accesses are still first touches to their own cache blocks after the same long interval. Their TLB hits therefore reflect prior same-page translation reuse rather than short reuse of the accessed cache blocks themselves. As a result, these later TLB-hit accesses preserve the same long-reuse implication as the preceding TLB miss within the same page. This masking effect shows that the long-reuse information exposed by the initial TLB miss can extend to later same-page accesses even when their translations hit in the TLB.

IV. MAVERICK

A. Design Overview

Maverick uses address-translation behavior as a unified signal to reduce memory latency. It consists of two complementary components: *Maverick-T*, which reduces translation latency, and *Maverick-OP*, which reduces the latency of subsequent instruction and data accesses. *Maverick-T* is invoked on TLB misses and shortens page walks by bypassing ineffective upper-level cache interactions and selectively overlapping leaf-level page-walk accesses predicted to go off chip. *Maverick-OP* predicts off-chip instruction/data accesses using a lightweight page prediction table that combines replay prediction after STLB misses with page-correlated first-touch prediction for later same-page accesses when the replayed access is confirmed to go off chip. Both components follow prior off-chip prediction studies [3], [6]: *Maverick* issues a speculative memory request before cache-hierarchy traversal while the regular request proceeds through the conventional path. If the prediction is correct, the regular request merges with the in-flight memory response. Otherwise, the speculative response is discarded without affecting correctness or polluting upper-level caches. Together, *Maverick* reduces latency across translation handling and demand-access execution by exploiting translation-driven locality information.

B. *Maverick-T*

Maverick-T is motivated by two properties of translation traffic: poor upper-level locality once translation reaches the cache hierarchy, and strong depth dependence in the behavior of leaf-level page-walk accesses. These observations motivate

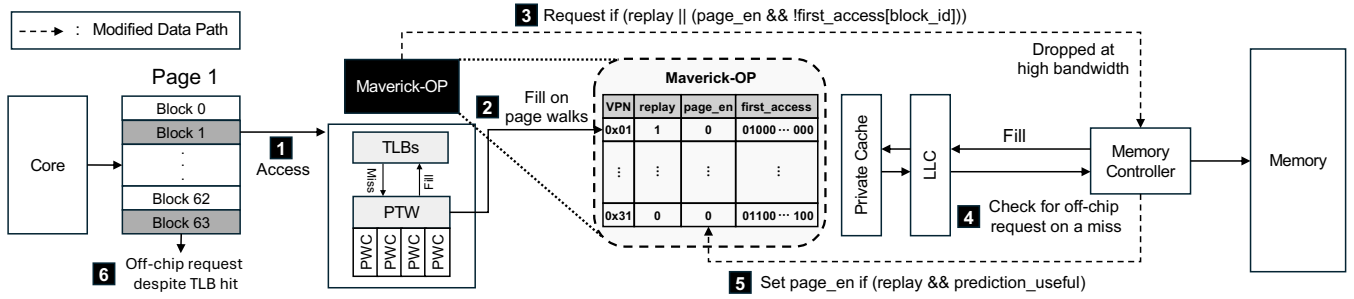


Fig. 10. Maverick-OP architecture. Translation outcomes seed replay-based prediction, which is extended to page-correlated first-touch accesses via a page-level prediction table, enabling speculative off-chip requests even on TLB hits.

tables. Each table is organized as a 16-set, 4-way associative structure (64 entries total). An entry is indexed by the virtual page number (VPN) and stores: (1) a VPN tag, (2) a per-block first-touch vector (64 bits), (3) a *replay* bit, (4) a *page_prediction_enable* bit, and (5) a valid bit. The *replay* bit is used to track whether an access originates from a replay following a deep translation miss (e.g., STLB miss), enabling Maverick-OP to identify seed accesses for page-level prediction. The *page_prediction_enable* bit serves as a page-level confidence indicator, allowing subsequent first-touch accesses within the same page to be predicted even if their translations hit in the TLB. The 64-bit first-touch vector tracks whether each cache block within the page has been accessed before, using the cache-block offset (e.g., VA[11:6]) as an index. This enables Maverick-OP to distinguish first-touch accesses from repeated accesses at cache-block granularity and avoid redundant speculation. To handle context switches in real systems, Maverick-OP can be implemented in two ways. The first option is to flush the page prediction tables on a context switch, conservatively preventing stale predictor state from being reused by a different address space. The second option is to tag each entry with an address-space identifier, such as ASID/PCID, similar to TLB entries. This avoids unnecessary flushing while preventing aliasing between different address spaces that may use the same VPN.

Operation Flow. Fig. 10 illustrates the hardware organization of Maverick-OP. Maverick-OP augments the translation and cache access path with a lightweight page-level predictor. For each demand instruction fetch or data load, Maverick-OP observes the access as it traverses the TLB hierarchy (1). On an STLB miss, it allocates the corresponding page entry in the prediction table and sets the *replay* bit to mark that the next demand access to this page follows a deep-translation replay (2). For every demand access, Maverick-OP also performs a parallel lookup in the page prediction table to decide whether to issue a speculative off-chip request (3). An access is predicted off-chip if the *replay* bit is set, or if the *page_prediction_enable* bit is set and the first-touch bit for the accessed cache line is zero. The corresponding first-touch bit is then set, and the *replay* bit is cleared to avoid repeated replay-triggered predictions. When predicted, Maverick-OP issues a speculative off-chip request as soon as the physical address is available, while the demand request

continues normally through L1, L2, and LLC. If the demand misses in the LLC, it checks for and merges with the in-flight speculative request, overlapping cache traversal with DRAM access (4). After the demand completes, Maverick-OP asserts the *page_prediction_enable* bit if the speculative request was useful, i.e., if the demand ultimately went off chip (5). This enables later TLB-hit accesses to the same page to still issue off-chip requests for first-touch cache lines (6), while incorrect predictions are safely discarded without polluting upper-level caches.

D. Datapath Design

Maverick introduces three mechanisms: translation bypassing, off-chip prediction for instruction/data accesses, and off-chip prediction for leaf-level page walks. This section describes how each mechanism is integrated into the memory-access datapath.

Bypassing for Translation Requests. Maverick-T modifies the datapath used by PTW-generated page-walk requests. In typical core organizations, private L1 instruction/data TLBs are placed close to the core, while the STLB is often located near the L2 cache. Since page walks are triggered only after an STLB miss, the PTW and PWCs are naturally placed near the STLB and the L2-side translation/cache interface. Recent processors therefore increasingly route MMU page-walk requests directly to the L2 cache [19], [30], [36], rather than through the L1 data cache. This organization also avoids exposing page-walk accesses to the L1 cache and can reduce page-walk side-channel leakage [52].

Maverick-T follows this trend and further bypasses the L2 cache for translation requests. This can be implemented in two ways. First, the PTW injection point can be changed from the L2 cache to the LLC, directly connecting the PTW to the LLC. This removes both L1 and L2 lookup/fill activity for translation traffic, but may add a PTW-to-LLC datapath. In many-core server systems, such a direct path can increase wiring, routing, and LLC-slice arbitration complexity, so its practicality depends on the floorplan and interconnect. Second, Maverick-T can keep the existing MMU-to-L2 connection and modify the L2 controller to treat PTW-generated page-walk requests as pass-through translation requests. Since these requests originate from the PTW rather than the core’s normal instruction-fetch or data-access pipeline, they can be distinguished using

the request source or type [36]. The L2 controller can then forward them to the LLC without L2 tag/data lookup, miss allocation, or cache line fill. This option avoids adding a separate long PTW-to-LLC wire while preserving the same logical behavior. Thus, the preferable implementation depends on the target SoC organization.

Off-Chip Prediction for Instruction and Data Requests.

Off-chip prediction in Maverick requires extensions to the conventional on-chip cache access datapath to support speculative off-chip requests. The design closely follows prior off-chip predictors such as Hermes [3] and Athena [6]. The prediction logic is implemented as an off-path side structure that operates in parallel with the regular demand access. Once a demand access is predicted to go off chip, Maverick issues a speculative memory request directly to the main memory controller upon physical address generation, bypassing the on-chip cache hierarchy. If the demand request later misses in the LLC, it merges with the in-flight speculative request. If it hits in the cache hierarchy, the speculative response is discarded without affecting upper-level caches. In our implementation, we also add a simple bandwidth-aware safeguard at the memory controller. When main-memory bandwidth usage exceeds 70% of the available bandwidth, speculative off-chip requests are dropped to avoid aggravating bandwidth contention. The performance benefit of off-chip prediction depends on the latency required to inject the speculative request into the memory controller, which we refer to as the speculative request issue latency. Although speculative requests bypass cache tag lookups and queueing delays, they still incur on-chip interconnect latency before reaching the memory controller. To account for different interconnect organizations, we model this issue latency as a configurable parameter and evaluate it across a range of values, following the methodology of Hermes [3].

Off-Chip Prediction for Leaf-Level Page Walks. Off-chip prediction for leaf-level page walks follows the same basic datapath as instruction/data off-chip prediction, except that the speculative request is initiated by the PTW rather than by the core’s instruction or data pipeline. However, supporting the prediction mechanism requires a small hardware extension for access tracking inside the page table walker as discussed above in Sec. IV-B. This access tracking can be implemented with a minor modification to the PTW control logic and a single LLC-miss bit returned with each memory request. Since a hardware page walk is a pointer-chasing process, the PTW already issues each page-walk memory request, waits for its response, and determines the next page-table address before continuing the walk. The PTW must already distinguish the current page-table level because the number of remaining pointer-chasing steps depends on which page-walk cache level hits or misses. Therefore, Maverick-T does not require a separate mechanism to identify the page-walk depth. The denominator counters can be updated when the PTW issues a memory request at the corresponding PWC-miss depth. The numerator counters require only one additional bit in the page-walk response, indicating whether the request missed in the LLC. When this bit is set for the corresponding leaf-level page-

TABLE I
STORAGE OVERHEAD OF MAVERICK

Structure	Description	Size
Instruction/Data Page Predictor	Two predictors (instruction + data), each with: 16 sets $\times$ 4 ways = 64 entries Each entry stores (bit-level): • Virtual page tag (VPN tag, $36 - 4 = 32$ bits) • Cache block first-touch vector (64 bits) • Replay-access bit (1 bit) • Page-prediction-enable bit (1 bit) • Valid bit (1 bit) Total 12,672 bits (≈ 1.54 KB)	~ 1.54 KB
	• Four 8-byte counters: C_{L2PWC_MISS} , $C_{L2_LEAF_OFF}$, C_{L3PWC_MISS} , $C_{L3_LEAF_OFF}$ (32B) • Threshold comparator T_1 (2B) Total: 34 B (≈ 0.04 KB)	
Page Walk Predictor		~ 0.04 KB
Total		~ 1.58 KB

walk request, the PTW increments the appropriate numerator counter stored in a small register. Thus, Maverick-T adds only lightweight counter-update logic to the existing PTW rather than introducing a separate prediction pipeline.

E. Design Considerations

Storage Overhead. Table I summarizes the storage overhead of Maverick. The instruction/data page predictor consists of two 16-set, 4-way predictor tables, one for instruction accesses and the other for data accesses, with 64 entries each. Each entry stores a virtual-page tag, a cache-block first-touch vector, a replay-access bit, a page-prediction-enable bit, and a valid bit, resulting in 99 bits per entry and 12,672 bits (approximately 1.54 KB) in total. In addition, Maverick employs a lightweight page-walk predictor for leaf-level off-chip prediction, composed of four 8-byte counters— C_{L2PWC_MISS} , $C_{L2_LEAF_OFF}$, C_{L3PWC_MISS} , and $C_{L3_LEAF_OFF}$ —and a 2-byte threshold comparator for T_1 , for a total of 34 B. Overall, Maverick requires only about 1.58 KB of additional storage, indicating that the proposed design achieves its performance benefits with minimal hardware overhead.

Page Sizes. Large pages are a widely used mechanism for reducing address-translation overhead by increasing TLB reach and reducing page-walk frequency. However, prior studies point out that reliably allocating large pages in datacenter systems remains challenging because 2 MB and 1 GB mappings require physically contiguous memory, which can be difficult to maintain under memory fragmentation and huge-page promotion [27], [50], [55]. To address these limitations, prior OS and allocator studies have explored hugepage-aware allocation, adaptive huge-page management, and mechanisms for preserving or recovering memory contiguity [18], [31], [53]. Maverick is complementary to these efforts, but large pages create a separate design tradeoff for Maverick because page-correlated prediction depends on identifying first-touch accesses within each translated page.

Specifically, cache-line-granular first-touch tracking is lightweight for 4KB pages but becomes expensive for 2MB pages. With 64B cache blocks, a 4KB page contains 64 cache lines, so a 64-bit first-touch vector is sufficient. Maverick’s instruction and data page predictors together require only about 1.54KB. In contrast, a 2MB page contains 32,768

cache lines, requiring a 32,768-bit vector to preserve the same granularity. With the same 128 total predictor entries, the first-touch vectors alone would require roughly 512KB, excluding tags and state bits. A cheaper alternative is to keep only 64 first-touch bits per large-page entry, but then each bit represents a 32KB region, or 512 cache lines, reducing distinguishable first-touch opportunities from 32,768 to 64. Thus, large-page support either incurs large metadata overhead or sacrifices Maverick’s cache-line-level coverage. It also requires page-size-aware indexing and different VPN tags, making a dedicated large-page prediction structure necessary. For this reason, the baseline Maverick design focuses on 4KB mappings and treats large-page support as an optional extension.

V. EXPERIMENTAL SETUP

TABLE II
SYSTEM CONFIGURATION

Component	Configuration
CPU Core	1 or 4 cores, 4GHz, 352-entry ROB
Branch Predictor	64KiB TAGE-SC-L [42]
BTB	16K-entry, 8-way set-associative
L1 ITLB	128-entry, 8-way, 1-cycle latency, 8-entry MSHR
L1 DTLB	64-entry, 4-way, 1-cycle latency, 8-entry MSHR
STLB	2048-entry, 16-way, 8-cycle latency, 16-entry MSHR
Page Walk Cache	5-level radix tree. L5 PWC: 2-entry, fully-associative; L4 PWC: 4-entry, fully-associative; L3 PWC: 8-entry, 2-way; L2 PWC: 32-entry, 4-way; 2-cycle access latency
L1 I-Cache	32KB, 8-way, 2-cycle access latency, 4-cycle round-trip latency, 8-entry MSHR, FDIP [22] prefetcher
L1 D-Cache	48KB, 12-way, 2-cycle access latency, 5-cycle round-trip latency, 16-entry MSHR, VIPT, Berti [34] prefetcher
L2 Cache	512KB, 8-way, 4-cycle access latency, 14-cycle round-trip latency, 32-entry MSHR, Pythia [5] prefetcher
LLC (per core)	2MB, 16-way, 14-cycle access latency, 41-cycle round-trip latency, 64-entry MSHR
DRAM	Single 64-bit channel (single-core), Two 64-bit channel (multi-core), 3200MT/s, tRP = tRCD = tCAS = 12.5ns

A. Evaluation Methodology

To evaluate Maverick, we use ChampSim [10], [15], a trace-driven cycle-level simulator modeling an out-of-order core and a three-level cache hierarchy with 64B blocks. Table II summarizes the baseline core, cache, and memory parameters, which follow an Icelake-like configuration [13]. We model a non-inclusive cache and TLB hierarchy, consistent with prior reverse-engineering studies [17], [48]. For hardware prefetching, we use Berti [34] in L1D and Pythia [5] in L2, following the 4th Data Prefetching Championship methodology [4], and apply no LLC prefetcher [28]. To capture realistic virtual memory behavior, we model a 5-level radix-tree page table, an x86 hardware page-table walker supporting up to four concurrent page walks, and multi-level page-walk caches. We also analyze the dynamic power consumption of the memory

hierarchy, including all cache levels and DRAM. The dynamic power of cache tag/data-array accesses and DRAM is obtained using McPAT [29] and Micron DRAM power calculator [33], respectively. We use a 22nm process technology for all estimations.

B. Evaluated Configurations

To evaluate Maverick, we compare it against three representative off-chip mechanisms.

- **POPET [3]:** POPET uses a hashed perceptron model with multiple program-context features, including PC-correlated cache-line/byte-offset features, first-access-related features, and recent load-PC history, to predict whether a load will go off chip. POPET targets only data loads.
- **HMP [54]:** HMP is a history-based predictor that uses past miss behavior, typically indexed by load PC, to identify loads likely to miss again and trigger early main-memory requests. Like POPET, HMP applies only to data loads.
- **TEMPO [9]:** TEMPO is a translation-triggered memory optimization that operates at the memory controller. It uses DRAM-resident leaf page-table accesses as a trigger for likely subsequent demand accesses to the same physical page. When a leaf page-table entry is fetched from DRAM, the memory controller extracts the corresponding physical page number and proactively activates the associated DRAM row, improving row-buffer locality for the following demand access. Originally proposed for data loads, we generalize the same trigger to instruction fetches.

For all evaluated mechanisms, we separately model the normal on-chip cache access latency and the speculative off-chip request issue latency. The cache access latency in Table II represents the modeled latency of traversing the on-chip cache hierarchy, including tag lookup and routing through the cache levels. In contrast, the speculative off-chip request issue latency represents the remaining routing and issue delay required to send a predicted off-chip request toward memory after bypassing the normal lookup path. We model this issue latency as 10 cycles for data loads and instruction fetches, and 5 cycles for page-walk requests. The shorter latency for page walks reflects Maverick-T’s direct forwarding of translation requests to the LLC. Thus, Maverick does not hide the full 20-cycle on-chip cache access latency. Instead, it hides the difference between the normal cache-traversal latency and the speculative issue latency.

C. Benchmarks and Workloads

We evaluate Maverick using 359 server-class traces provided by Google for the 4th Data Prefetching Championship [4]. These traces are anonymized Google warehouse-scale computer (WSC) workload traces. The traces are grouped into 12 anonymized workload families: arizona (6 traces), bravo (3), charlie (22), delta (4), merced (89), sierra.a.3 (17), sierra.a.4 (24), sierra.a.6 (18), tahoe (40), tango (51), whiskey (2), and yankee (83). Since the workload names are anonymized, they do not reveal the underlying production services, but they provide a broad set of server-class

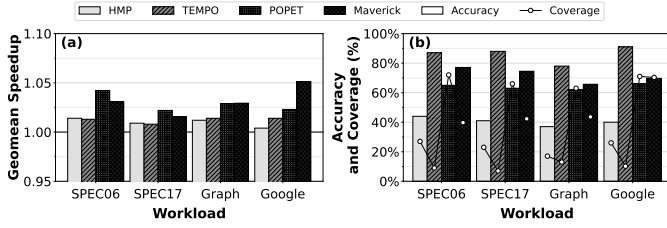


Fig. 11. Cross-workload evaluation of Maverick and prior off-chip predictors. (a) Geomean speedup across SPEC CPU2006, SPEC CPU2017, graph, and Google WSC workloads. (b) Prediction accuracy (bars) and coverage (lines) for data loads across the same workload suites.

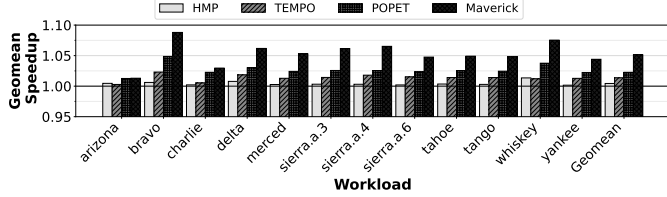


Fig. 12. Speedup across Google server workload suites for HMP, TEMPO, POPET, and Maverick. Bars show the geometric mean speedup of all traces in each suite, normalized to the baseline.

instruction and memory-access behaviors for evaluating data-access and translation bottlenecks. To further examine the robustness of Maverick beyond the Google WSC traces, we also evaluate Maverick on additional workload suites, including SPEC CPU2006 [45] available in ChampSim [10], SPEC CPU2017 [46], and graph workloads [2], [43] that were used in the 4th Data Prefetching Championship [4], [12]. Following prior off-chip prediction studies such as Hermes and Athena [3], [6], we focus on memory-intensive traces whose baseline LLC demand MPKI is at least 3 for these workloads.

Simulation Methodology. For single-core experiments, we warm up each core for 50M instructions and measure performance over the following 200M instructions, following prior work [4], [50], [51]. For multi-core experiments, we evaluate heterogeneous 4-core mixes, where each core runs a different trace randomly drawn from the workload pool. We follow the DPC-4 multi-core (MC) scoring methodology [4], [12]: for each mix, we compute harmonic speedup for both the baseline and the evaluated configuration using per-core shared IPCs normalized to the corresponding single-core baseline IPC, and report overall MC speedup as the non-weighted geometric mean of per-mix speedup ratios relative to the baseline. Each class contains 100 workload mixes. In multi-core simulations, each core is warmed up for 50M instructions and measured over the subsequent 100M instructions. Cores that finish warmup early stall until all others complete warmup, so measurement begins from aligned execution points, and benchmarks that finish early are restarted until all cores complete the measurement window [3], [5], [23], [34], [37], [38].+

VI. EVALUATION

A. Performance Analysis

We first examine Maverick across different workload classes to characterize where its translation-driven mechanisms provide the greatest opportunity. Unless otherwise noted, all single-core speedups in this section are normalized to the prefetcher-enabled baseline with FDIP, Berti, and Pythia enabled, without off-chip prediction. As shown in Fig. 11(a), Maverick improves performance across all evaluated workload suites, achieving geomean speedups of 3.06% on SPEC CPU2006, 1.53% on SPEC CPU2017, 2.93% on graph workloads, and 5.12% on the Google WSC workloads. The larger benefit on Google WSC mainly stems from two characteristics of server workloads. First, unlike Google WSC, where data, instruction, and translation accesses all contribute substantially to off-chip stalls, approximately 95% of off-chip requests in the additional workload suites come from data accesses. This concentrates most off-chip optimization opportunities on the data path, limiting the additional benefit available from Maverick’s broader instruction and translation side mechanisms. Second, many benchmarks in these suites use relatively small datasets, allowing their translation working sets to be largely covered by the TLB hierarchy and resulting in fewer TLB misses. This further reduces the opportunities for both Maverick-T’s page-walk latency reduction and Maverick-OP’s translation-guided off-chip prediction. In contrast, the large instruction, data, and translation working sets of Google WSC expose substantially more optimization opportunities across all three access paths.

Fig. 11(b) further examines Maverick’s prediction quality across workload classes. Because off-chip requests in SPEC06, SPEC17, and Graph workloads are dominated by data loads (approximately 95%), we focus this cross-workload comparison on data-load predictions. Maverick achieves accuracies of 65.7%, 77.1%, and 74.5% on graph workloads, SPEC06, and SPEC17, respectively, comparable to 69.7% on Google WSC. In contrast, their coverage is 43.6%, 39.7%, and 42.3%, respectively, compared with 70.3% on Google WSC. The lower coverage is mainly due to their smaller translation working sets, which are more effectively covered by the TLB hierarchy and therefore generate fewer TLB misses and fewer translation-derived prediction opportunities. These results indicate that the smaller performance gains on the additional workloads primarily stem from fewer prediction opportunities rather than degraded prediction accuracy. Taken together, these characteristics motivate Maverick’s primary focus on server workloads, where large instruction, data, and translation working sets expose substantial optimization opportunities across all three access paths. We therefore focus the remainder of our detailed evaluation on the 359 Google WSC traces and later analyze Maverick’s accuracy and coverage across data, instruction, and translation accesses.

Fig. 12 reports performance across the 12 Google WSC workload suites. Across all suites, Maverick consistently delivers the highest speedup, achieving a geomean improvement

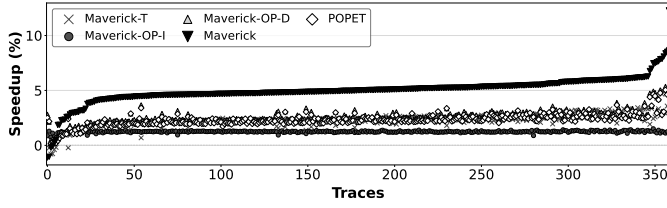


Fig. 13. Per-trace speedup breakdown of Maverick across 359 server traces, showing translation (Maverick-T), instruction-side (Maverick-OP-I), and data-side (Maverick-OP-D) contributions. POPET is shown for comparison.

of 5.12% over the baseline, compared with 2.27% for POPET, 1.36% for TEMPO, and 0.44% for HMP. Maverick achieves a 2.72% geomean speedup over POPET, the best-performing prior off-chip predictor. The gains are particularly pronounced in memory-intensive suites such as *bravo* (8.80%), *whiskey* (7.54%), *sierra.a.4* (6.52%), *delta* (6.18%), and *sierra.a.3* (6.16%), where long-latency translation and demand accesses are both frequent.

Fig. 13 breaks down Maverick’s per-trace speedup across all 359 server traces into translation, data, and instruction components. On average, Maverick-T contributes 2.29% geomean speedup, while Maverick-OP contributes 2.38% from data accesses and 1.24% from instruction accesses; together, these components yield an overall geomean speedup of 5.12%. These results show that a single translation-driven scheme can be applied effectively across all three access types. In server workloads, translation, data, and instruction accesses can all become bottlenecks, and Maverick improves performance by using translation-derived signals to reduce latency in all three paths rather than optimizing only one in isolation. For comparison, POPET, the strongest prior work, achieves 2.27% geomean speedup but targets only data loads. Maverick outperforms prior work by using a unified translation-driven framework that simultaneously improves translation handling and broadens off-chip prediction coverage across all memory access types.

We also observe a small number of cases where Maverick-T reduces the benefit of the full design. This behavior occurs only in certain execution phases within 6 out of 359 traces, rather than being a persistent characteristic of a specific workload. These phases arise when translation entries already achieve high hit rates in the L1 and L2 caches. In such cases, bypassing can remove useful translation residency and slightly offset the gains from Maverick-OP; however, for the vast majority of phases and traces, Maverick-T remains beneficial by reducing cache traversal latency and cache pollution from low-locality translation entries.

B. Detailed Analysis

Prediction Accuracy and Coverage. Fig. 14 compares the accuracy and coverage of representative off-chip predictors. HMP achieves only 41% accuracy and 27% coverage on data accesses, indicating that simple miss-history tracking is neither accurate nor broad enough to capture many off-chip opportunities. POPET improves this balance on the data path, reaching

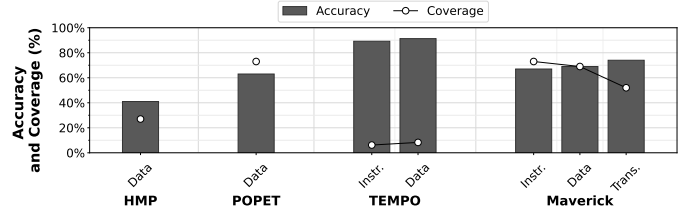


Fig. 14. Accuracy (bars) and coverage (line) of off-chip predictors across access types. HMP and POPET apply to data accesses only, TEMPO to instruction and data accesses, and Maverick to instruction, data, and translation accesses.

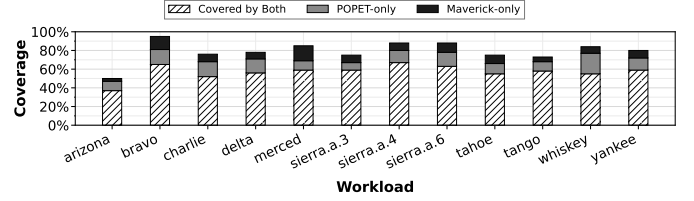


Fig. 15. Coverage overlap between Maverick and POPET for correctly predicted off-chip accesses for data loads.

63% accuracy and 73% coverage, but remains limited to data loads. TEMPO, in contrast, attains very high accuracy for both instruction and data accesses (89.2% and 91.3%, respectively), yet its coverage is extremely low at only 6.2% and 8.3%, since it is triggered only by a narrow subset of translation events that reach the main memory. Maverick achieves both high accuracy and broad coverage by separating when page-level prediction is enabled from how far it is extended. For accuracy, Maverick first uses an STLB-miss replay as a seed prediction and enables page-correlated prediction only when this replay is confirmed to go off chip. This ensures that later predictions are made only for pages that have already exhibited long-latency behavior. For coverage, Maverick then extends this information to later first-touch accesses within the same page, including accesses whose translations hit in the TLB. This allows Maverick to capture off-chip opportunities that replay-only prediction would otherwise miss.

Fig. 15 further isolates Maverick’s incremental contribution over POPET. To avoid perturbing execution, both predictors are evaluated in prediction-only mode, where they generate predictions without issuing early off-chip requests. Correctly predicted accesses are classified as predicted by both, only by POPET, or only by Maverick. The results show substantial overlap but also distinct coverage. POPET-only accesses arise from its rich program-context features, while Maverick-only accesses reflect additional translation-correlated, page-level opportunities. This complementary coverage suggests that POPET and Maverick can be synergistic, but simply adding translation outcomes as POPET features is insufficient. A POPET variant augmented with DTLB hit/miss, STLB hit/miss, and page-walk depth improves accuracy by 4.7% but provides little additional coverage. Achieving Maverick’s broader coverage requires per-page first-access tracking, while

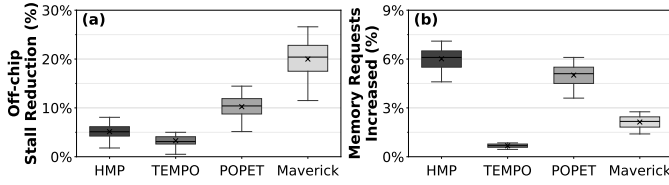


Fig. 16. (a) Reduction in off-chip stall cycles. (b) Increase in memory requests for different off-chip predictors.

usefulness tracking of replayed TLB-missing accesses further improves accuracy.

Memory Latency & Bandwidth. Fig. 16 quantifies Maverick’s tradeoff between latency reduction and additional memory traffic. Fig. 16(a) shows the reduction in stall cycles due to off-chip requests, while Fig. 16(b) reports the increase in memory requests. Maverick achieves the largest stall reduction among all schemes, with an average reduction of 20.99%, compared with 10.24% for POPET, 5.14% for HMP, and 3.99% for TEMPO. This result is consistent with Maverick’s broader prediction coverage across translation, data, and instruction accesses, which allows it to overlap a larger fraction of long-latency memory operations. At the same time, Maverick increases memory requests by only 2.13% on average, which is substantially lower than HMP (6.01%) and POPET (5.01%), despite delivering much larger latency reduction. This modest bandwidth cost comes in part from Maverick-T’s bypassing mechanism. By preventing translation blocks from occupying L1 and L2, Maverick-T reduces upper-level cache pollution and preserves more space for instruction and data blocks, which in turn increases their private-cache hit rates and reduces the number of demand accesses that fall through to the LLC. In our evaluation, applying Maverick-T’s bypassing alone reduces LLC MPKI by about 3.6%, confirming that the reduced cache pollution translates into fewer lower-level misses. As a result, Maverick can achieve strong off-chip stall reduction without incurring the memory-request growth typically associated with aggressive off-chip speculation.

Dynamic Power Consumption. Maverick also reduces on-chip dynamic power while incurring only a modest increase in memory-side power. Due to Maverick-T’s bypassing mechanism, translation requests no longer access the L1 data cache or L2, reducing their dynamic power by 3.65% and 3.27%, respectively. Maverick also lowers LLC dynamic power by 5.46%. This LLC reduction is indirect: by preventing translation pollution in L1 and L2, Maverick increases instruction and data hit rates in the upper levels and reduces the number of accesses that fall through to the LLC. Although Maverick increases memory dynamic power by about 2.9% due to additional speculative off-chip requests, the overall runtime dynamic power consumption still decreases by about 0.72%. As an additional check for high-bandwidth memory, DRAMPower-5.0 [47] based estimate shows that Maverick increases average DRAM power by only 1.26% at 12800 MT/s.

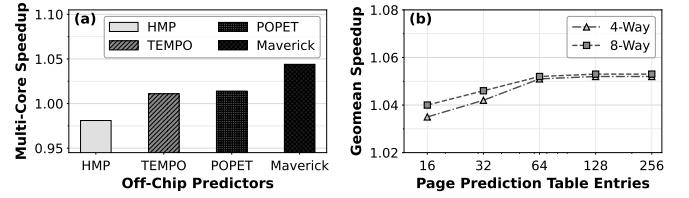


Fig. 17. (a) Multi-core speedup under the DPC-4 MC scoring methodology. (b) Page prediction table size sensitivity.

C. Sensitivity Study

We next evaluate Maverick’s robustness across a range of architectural and system parameters. Unless otherwise noted, the sensitivity studies use the same Google WSC workload set and simulation methodology described in Section V-C. Single-core results report geomean speedup across all 359 traces relative to the prefetcher-enabled baseline, while the multi-core analysis follows the DPC-4 MC methodology described in Section V-C.

Multi-Core Analysis. Fig. 17(a) reports multi-core performance under the DPC-4 MC scoring methodology introduced in Sec. V-C. Maverick achieves the highest multi-core speedup at 1.043 $\times$, outperforming POPET (1.013 $\times$), TEMPO (1.011 $\times$), and HMP (0.981 $\times$). Although speculative off-chip mechanisms can become less effective in multi-core settings due to increased contention for shared memory resources, Maverick remains robust because its memory-request overhead is relatively modest. In particular, both TEMPO and Maverick continue to operate effectively in the multi-core regime because they do not excessively inflate memory traffic. For Maverick, this property is further aided by Maverick-T’s bypassing mechanism, which reduces translation pollution in upper caches and limits unnecessary fall-through accesses to lower levels. As a result, Maverick preserves its advantage even under shared-memory interference, whereas HMP fails to deliver a net gain and POPET provides only limited improvement.

Page Prediction Table Entries. Fig. 17(b) shows the sensitivity of Maverick to the number of page prediction table entries. Maverick’s performance improves as the table size increases from 16 to 64 entries, because a larger table can retain more page-level replay and first-touch information and reduce thrashing from frequent entry replacement. However, the benefit largely saturates beyond 64 entries, with 128 and 256 entries providing only marginal additional speedup. Higher associativity also helps when the table is small: the 8-way configuration improves performance over 4-way at 16 and 32 entries by reducing conflict-induced thrashing. Once the table has 64 or more entries, the gap between 4-way and 8-way becomes negligible. Therefore, we choose a 64-entry, 4-way page prediction table as the area/performance sweet spot.

Memory Bandwidth. Fig. 18(a) shows the sensitivity of Maverick to available memory bandwidth and the impact of the bandwidth-aware safeguard. As bandwidth increases

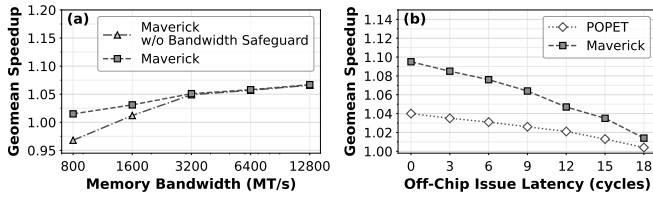


Fig. 18. (a) Geomean speedup as a function of available memory bandwidth, with and without the bandwidth safeguard. (b) Geomean speedup as a function of speculative off-chip request issue latency.

from 800MT/s to 12800MT/s, Maverick’s speedup increases from $1.014\times$ to $1.067\times$, showing that Maverick can further benefit when additional memory bandwidth is available. More importantly, the bandwidth safeguard is critical under constrained bandwidth. We set the bandwidth-safeguard threshold to 70% as discussed earlier, which provided the best overall performance across configurations in a sweep from 50% to 100% in 5% increments. Without the safeguard, speculative off-chip requests can consume scarce memory bandwidth and occupy memory-system resources, delaying other demand requests on the critical path. As a result, Maverick without the safeguard falls below the baseline at 800MT/s, achieving only $0.968\times$ speedup. In contrast, the full Maverick design still achieves $1.014\times$ speedup by dropping 67.3% of speculative off-chip requests when main-memory bandwidth utilization becomes high. The fraction of dropped speculative off-chip requests decreases as bandwidth increases: 67.3%, 35.1%, 8.1%, 5.5%, and 2.1% at 800, 1600, 3200, 6400, and 12800MT/s, respectively. As bandwidth increases, the gap between the two configurations narrows, and they converge around 3200MT/s, indicating that the safeguard mainly protects performance under bandwidth pressure.

Off-chip Request Issue Latency. Fig. 18(b) shows sensitivity to speculative off-chip request issue latency. As described in Section V-B, we use separate issue latencies for instruction/data and translation off-chip requests, setting the baseline to 10 cycles and 5 cycles, respectively. As shown in Table II, a regular instruction/data request traverses the L1, L2, and LLC access path, which in our model accounts for 20 cycles of cache-access latency ($2 + 4 + 14$ cycles), before reaching memory. A speculative off-chip request bypasses this cache hierarchy and the associated on-chip queueing delays, but still incurs routing latency through the on-chip interconnect before arriving at the memory controller. Therefore, assigning a 10-cycle issue latency to speculative instruction/data requests is a deliberately conservative approximation. To evaluate robustness, we vary the instruction/data issue latency from 0 to 18 cycles, while setting the translation issue latency to half of that value. As expected, the benefit decreases as issue latency grows, since less of the off-chip access can be overlapped with on-chip lookup. Even at 18 cycles, however, Maverick still delivers a $1.014\times$ speedup. This residual benefit comes from two sources. First, Maverick-T’s translation bypassing still removes ineffective upper-level cache accesses on the translation path, so part of its benefit is independent of specu-

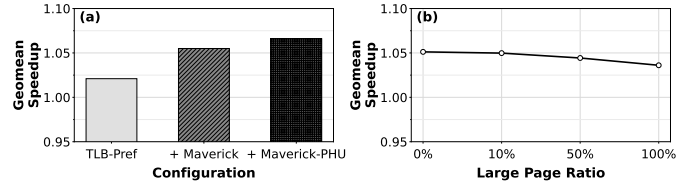


Fig. 19. (a) Sensitivity of Maverick to a TLB prefetcher. (b) Sensitivity to the fraction of 2 MB large-page mappings with region-based first-touch tracking.

lative overlap latency. Second, even when speculative requests are injected later, they still bypass the normal cache-hierarchy traversal and its associated queueing delays, preserving part of the overlap opportunity. These results show that Maverick remains beneficial even under pessimistic speculative-request injection latency.

TLB Prefetcher Sensitivity. We also evaluate Maverick with a hardware TLB prefetcher [24] to understand its interaction with MMU-side prefetching. When both mechanisms operate together, two effects arise. First, a successful TLB prefetch can convert a future TLB miss into a hit, partially overlapping with Maverick-T’s opportunity to reduce page-walk latency. Second, because Maverick-OP updates its page-prediction table on TLB-miss events, TLB prefetching can hide these events and reduce Maverick-OP’s prediction coverage. To recover this lost learning opportunity, we add prefetch-hit update (PHU), which updates Maverick-OP’s table when a demand access hits on a prefetched TLB entry. As shown in Fig. 19(a), TLB-Pref alone improves performance by 2.1%, while adding Maverick increases the speedup to 5.5%. With PHU, the geomean speedup further increases to 6.6%, by recovering table-update opportunities hidden by TLB-prefetch hits. These results show that Maverick still provides substantial benefits on top of TLB prefetching, and that PHU enables Maverick to better coexist with aggressive TLB/MMU prefetchers.

Large-Page Sensitivity. We further evaluate Maverick by varying the fraction of memory mapped with 2 MB large pages from 0% to 100%, with each result normalized to the prefetcher-enabled baseline using the same large-page ratio. For large-page mappings, Maverick divides each 2 MB page into 64 32 KB regions and tracks one first-touch bit per region, preserving the original 64-bit metadata per predictor entry at the cost of coarser prediction granularity. As shown in Fig. 19(b), Maverick’s speedup gradually decreases from 5.1% with only 4 KB pages to 5.0%, 4.4%, and 3.6% at large-page ratios of 10%, 50%, and 100%, respectively. This reduction occurs because large pages increase TLB reach and reduce page walks, while region-level tracking merges multiple cache-line first touches and therefore lowers Maverick-OP’s prediction opportunities. Nevertheless, Maverick retains a 3.6% speedup even when all mappings use large pages. We randomly select large-page regions in this experiment to isolate sensitivity to large-page coverage; real systems promote pages non-uniformly according to physical contiguity, fragmentation, OS promotion policies, and workload behavior, so the exact performance impact may differ depending on which regions

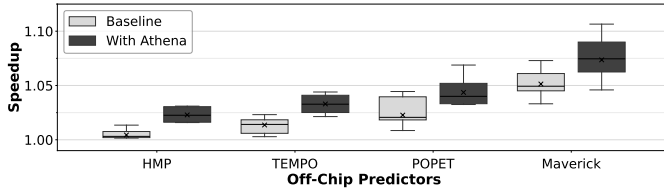


Fig. 20. Sensitivity to Athena-based prefetcher/OCP coordination across different off-chip predictors.

are mapped with large pages.

Impact of Prefetcher/OCP Coordination. Fig. 20 evaluates Maverick with Athena [6], a recently proposed RL-based framework that coordinates data prefetchers and off-chip predictors for data loads. Athena is not a standalone off-chip predictor; instead, it dynamically controls whether to enable the prefetcher, the off-chip predictor, both, or neither, while also adjusting prefetcher aggressiveness based on runtime system behavior. Across all evaluated off-chip predictors, Athena improves performance by reducing harmful interactions between prefetching and speculative off-chip requests. Maverick also benefits from Athena, showing that Maverick is complementary to prefetcher/OCP coordination. Importantly, Maverick achieves the highest performance both with and without Athena, because its translation-driven predictions expose additional off-chip opportunities beyond conventional data-only predictors. These results show that Maverick’s benefits are orthogonal to Athena’s coordination policy and can be further amplified when combined with dynamic prefetcher/OCP control.

VII. DISCUSSION

Negative Side Effects of Maverick-T Bypassing. Maverick-T’s L1/L2 bypass is based on the observation that translation requests reaching the cache hierarchy generally exhibit low upper-level locality. However, in rare execution phases, page-table entries may retain sufficient locality in the upper-level caches, in which case bypassing can remove useful translation residency and increase page-walk latency. We observe this behavior in only 6 out of the 359 Google WSC traces, indicating that it is a rare and phase-specific effect rather than a persistent characteristic of a particular workload. Despite this temporary downside, the full Maverick design still achieves meaningful overall speedup on these traces because the benefits from Maverick-OP and from other execution phases outweigh the loss from bypassing. A potential mitigation is an adaptive bypass policy using a small saturating counter that detects sustained upper-cache utility and temporarily disables bypassing. Because this decision is based on the observed benefit of upper-level cache accesses rather than a fixed cache configuration, such an adaptive policy could also make Maverick-T less sensitive to changes in cache size, associativity, or other hierarchy parameters that affect translation locality. However, dynamically switching between bypassed and conventional cache paths requires additional care for coherence. Once conventional lookup-and-fill is enabled, page-table entries may again reside in the upper-level caches,

and re-enabling bypassing cannot safely assume that no valid translation blocks remain there. In particular, this adaptive policy is not directly applicable to the implementation that connects the PTW directly to the LLC unless the conventional L2 path is also retained and appropriate coherence or invalidation mechanisms are provided. The L2 pass-through implementation is more amenable to such adaptation, although it would still require a safe transition mechanism before bypassing is re-enabled. We leave the design and evaluation of such an adaptive coherent bypass mechanism for future work.

VIII. CONCLUSION

This paper presented *Maverick*, a translation-driven mechanism for reducing memory latency in server workloads. We showed that address translation is not merely a source of overhead, but also a useful architectural signal: translation requests that reach the cache hierarchy already form a low-locality tail after TLB/PWC filtering, and translation behavior strongly correlates with that of the dependent memory requests that follow it. By exploiting these properties, *Maverick* reduces latency across translation, instruction, and data accesses together. This is particularly important in server workloads, where all three can become major performance bottlenecks, suggesting that address translation can serve as a unified signal for mitigating memory latency more broadly.

REFERENCES

- [1] T. W. Barr, A. L. Cox, and S. Rixner, “Translation caching: skip, don’t walk (the page table),” *ACM SIGARCH Computer Architecture News*, vol. 38, no. 3, pp. 48–59, 2010.
- [2] S. Beamer, K. Asanović, and D. Patterson, “The GAP Benchmark Suite,” *arXiv preprint arXiv:1508.03619*, 2015. [Online]. Available: <https://arxiv.org/abs/1508.03619>
- [3] R. Bera, K. Kanellopoulos, S. Balachandran, D. Novo, A. Olgun, M. Sadrosadat, and O. Mutlu, “Hermes: Accelerating long-latency load requests via perceptron-based off-chip load prediction,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 1–18.
- [4] R. Bera, K. Kanellopoulos, and O. Mutlu, “The 4th data prefetching championship (dpc-4),” <https://sites.google.com/view/dpc4-2026/>, 2026.
- [5] R. Bera, K. Kanellopoulos, A. Nori, T. Shahroodi, S. Subramoney, and O. Mutlu, “Pythia: A customizable hardware prefetching framework using online reinforcement learning,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1121–1137.
- [6] R. Bera, Z. Lang, C. Hengartner, K. Kanellopoulos, R. Kumar, M. Sadrosadati, and O. Mutlu, “Athena: Synergizing data prefetching and off-chip prediction via online reinforcement learning,” *arXiv preprint arXiv:2601.17615*, 2026.
- [7] R. Bhargava, B. Serebrin, F. Spadini, and S. Manne, “Accelerating two-dimensional page walks for virtualized systems,” in *Proceedings of the 13th international conference on Architectural support for programming languages and operating systems*, 2008, pp. 26–35.
- [8] A. Bhattacharjee, “Large-reach memory management unit caches,” in *Proceedings of the 46th Annual IEEE/ACM International Symposium on Microarchitecture*, 2013, pp. 383–394.
- [9] —, “Translation-triggered prefetching,” in *Proceedings of the Twenty-Second International Conference on Architectural Support for Programming Languages and Operating Systems*, 2017, pp. 63–76.
- [10] ChampSim Contributors, “ChampSim,” <https://github.com/ChampSim/ChampSim>, accessed: 2026-07-02.
- [11] D. Chasapis, G. Vavoulitis, D. A. Jiménez, and M. Casas, “Instruction-aware cooperative tlb and cache replacement policies,” in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 2025, pp. 619–636.

- [12] CMU SAFARI Research Group, “DPC4: Data Prefetching Championship 4,” <https://github.com/CMU-SAFARI/DPC4>, 2024, gitHub repository, accessed 2026-03-19.
- [13] I. Cutress, “The ice lake benchmark preview: Inside intel’s 10nm,” <https://www.anandtech.com/show/14664/testing-intel-ice-lake-10nm2>, Aug. 2019.
- [14] M. Ferdman, A. Adileh, O. Kocberber, S. Volos, M. Alisafaei, D. Jevdjic, C. Kaynak, A. D. Popescu, A. Ailamaki, and B. Falsafi, “Clearing the clouds: a study of emerging scale-out workloads on modern hardware,” *ACM SIGPLAN Notices*, vol. 47, no. 4, pp. 37–48, 2012.
- [15] N. Gober, G. Chacon, L. Wang, P. V. Gratz, D. A. Jimenez, E. Teran, S. Pugsley, and J. Kim, “The championship simulator: Architectural simulation for education and competition,” *arXiv preprint arXiv:2210.14324*, 2022.
- [16] B. R. Godala, S. P. Ramesh, G. A. Pokam, J. Stark, A. Seznec, D. Tullsen, and D. I. August, “Pdip: Priority directed instruction prefetching,” in *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 2*, 2024, pp. 846–861.
- [17] B. Gras, K. Razavi, H. Bos, and C. Giuffrida, “Translation leak-aside buffer: Defeating cache side-channel protections with tlb attacks,” in *27th USENIX Security Symposium (USENIX Security 18)*, 2018, pp. 955–972.
- [18] M. Hunter and I. Kabiljo, “Beyond malloc efficiency to fleet efficiency: a hugepage-aware memory allocator,” in *Proceedings of the 15th USENIX Symposium on Operating Systems Design and Implementation*, 2021, pp. 257–273.
- [19] Intel Corporation, *Intel 64 and IA-32 Architectures Software Developer’s Manual*. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [20] —, *Intel® 64 and IA-32 Architectures Performance Monitoring Events*, revision 1.0 ed., 2017, document Number: 335279-001. [Online]. Available: <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>
- [21] —, *Intel® 64 and IA-32 Architectures Optimization Reference Manual*, 2023, available at <https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-optimization-manual.pdf>.
- [22] Y. Ishii, J. Lee, K. Nathella, and D. Sunwoo, “Re-establishing fetch-directed instruction prefetching: An industry perspective,” in *2021 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*. IEEE, 2021, pp. 172–182.
- [23] D. A. Jiménez and E. Teran, “Multiperspective reuse prediction,” in *Proceedings of the 50th Annual IEEE/ACM International Symposium on Microarchitecture*, 2017, pp. 436–448.
- [24] G. B. Kandiraju and A. Sivasubramaniam, “Going the distance for tlb prefetching: An application-driven study,” *ACM SIGARCH Computer Architecture News*, vol. 30, no. 2, pp. 195–206, 2002.
- [25] S. Kanev, J. P. Darago, K. Hazelwood, P. Ranganathan, T. Moseley, G.-Y. Wei, and D. Brooks, “Profiling a warehouse-scale computer,” in *Proceedings of the 42nd annual International Symposium on Computer Architecture*, 2015, pp. 158–169.
- [26] J. Kwon, Y. Lee, J. Kim, E. Jang, H. Kal, and W. W. Ro, “Garibaldi: A pairwise instruction-data management for enhancing shared last-level cache performance in server workloads,” in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, 2025, pp. 1049–1063.
- [27] Y. Kwon, H. Yu, S. Peter, C. J. Rossbach, and E. Witchel, “Coordinated and efficient huge page management with ingens,” in *Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation*, 2016, pp. 705–721.
- [28] C. Lam, “Hot chips: Arm’s neoverse v2,” <https://chipsandcheese.com/p/hot-chips-2023-arms-neoverse-v2>, 2023.
- [29] S. Li, J. H. Ahn, R. D. Strong, J. B. Brockman, D. M. Tullsen, and N. P. Jouppi, “Mcpat: An integrated power, area, and timing modeling framework for multicore and manycore architectures,” in *Proceedings of the 42nd annual IEEE/ACM International Symposium on Microarchitecture*, 2009, pp. 469–480.
- [30] A. Ltd. Hardware page table translation. [Online]. Available: <https://developer.arm.com/documentation/ddi0333/h/memory-management-unit/hardware-page-table-translation?lang=en>
- [31] M. Maas, C. Kennelly, K. Nguyen, D. Gove, K. S. McKinley, and P. J. Turner, “Adaptive huge-page subrelease for non-moving memory allocators in warehouse-scale computers,” in *Proceedings of the 2021 ACM SIGPLAN International Symposium on Memory Management*, 2021, pp. 28–38.
- [32] S. Mahar, H. Wang, W. Shu, and A. Dhanotia, “Workload behavior driven memory subsystem design for hyperscale,” *arXiv preprint arXiv:2303.08396*, 2023.
- [33] Micron Technology, Inc., “Micron dram power calculator,” <https://www.micron.com/content/dam/micron/global/secure/products/technical-note/dram/t4007-ddr4-power-calculation.pdf>, 2015.
- [34] A. Navarro-Torres, B. Panda, J. Alastruey-Benedé, P. Ibáñez, V. Viñals-Yúfera, and A. Ros, “Berti: an accurate local-delta data prefetcher,” in *2022 55th IEEE/ACM International Symposium on Microarchitecture (MICRO)*. IEEE, 2022, pp. 975–991.
- [35] S. Oh, M. Xu, T. A. Khan, B. Kasicki, and H. Litz, “Udp: Utility-driven fetch directed instruction prefetching,” in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2024, pp. 1188–1201.
- [36] OpenXiangShan, “Xiangshan: Pagetablewalker.scala,” <https://github.com/OpenXiangShan/XiangShan/blob/mdp-train-fix/src/main/scala/xiangshan/cache/mmu/PageTableWalker.scala>, gitHub repository; branch mdp-train-fix.
- [37] S. Pakalapati and B. Panda, “Bouquet of instruction pointers: Instruction pointer classifier-based spatial hardware prefetching,” in *2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2020, pp. 118–131.
- [38] B. Panda, “Clip: Load criticality based data prefetching for bandwidth-constrained many-core systems,” in *Proceedings of the 56th Annual IEEE/ACM International Symposium on Microarchitecture*, 2023, pp. 714–727.
- [39] C. H. Park, I. Vougioukas, A. Sandberg, and D. Black-Schaffer, “Every walk’s a hit: making page walks single-access cache hits,” in *Proceedings of the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2022, pp. 128–141.
- [40] G. Reinman, B. Calder, and T. Austin, “Fetch directed instruction prefetching,” in *MICRO-32. Proceedings of the 32nd Annual ACM/IEEE International Symposium on Microarchitecture*. IEEE, 1999, pp. 16–27.
- [41] A. Ros and A. Jimborean, “The entangling instruction prefetcher,” *IEEE Computer Architecture Letters*, vol. 19, no. 2, pp. 84–87, 2020.
- [42] A. Seznec, “Tage-sc-l branch predictors again,” in *5th JILP Workshop on Computer Architecture Competitions (JWAC’5): Championship Branch Prediction (CBP-5)*, 2016.
- [43] J. Shun and G. E. Blelloch, “Ligra: A Lightweight Graph Processing Framework for Shared Memory,” in *Proceedings of the ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, ser. PPOPP, 2013, pp. 135–146.
- [44] A. Sriraman, A. Dhanotia, and T. F. Wenisch, “Softsku: Optimizing server architectures for microservice diversity @Scale,” in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 513–526.
- [45] Standard Performance Evaluation Corporation, “SPEC CPU 2006,” 2006, accessed: 2026-06-08. [Online]. Available: <https://www.spec.org/cpu2006/>
- [46] —, “Spec cpu 2017,” <https://www.spec.org/cpu2017/>, 2017.
- [47] L. Steiner, T. Psota, M. Mörz, D. Christ, M. Jung, and N. Wehn, “Dram-power 5: An open-source power simulator for current generation dram standards,” in *Proceedings of the Rapid Simulation and Performance Evaluation for Design*, 2025, pp. 8–16.
- [48] A. Tatar, D. Trujillo, C. Giuffrida, and H. Bos, “TLB;DR: Enhancing tlb-based attacks with tlb desynchronized reverse engineering,” in *31st USENIX Security Symposium (USENIX Security 22)*, 2022, pp. 989–1007.
- [49] S. van Schaik, K. Razavi, B. Gras, H. Bos, and C. Giuffrida, “Reverse engineering hardware page table caches using side-channel attacks on the mmu,” *Vrije Universiteit Amsterdam, Tech. Rep.*, 2017.
- [50] G. Vavouliotis, L. Alvarez, B. Grot, D. Jiménez, and M. Casas, “Morgan: A composite instruction tlb prefetcher,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 1138–1153.
- [51] G. Vavouliotis, L. Alvarez, V. Karakostas, K. Nikas, N. Koziris, D. A. Jiménez, and M. Casas, “Exploiting page table locality for agile tlb prefetching,” in *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 2021, pp. 85–98.
- [52] A. Wang, B. Chen, Y. Wang, C. W. Fletcher, D. Genkin, D. Kohlbrenner, and R. Paccagnella, “Peek-a-walk: Leaking secrets via page walk side

- channels,” in *2025 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2025, pp. 3534–3548.
- [53] Z. Yan, D. Lustig, D. Nellans, and A. Bhattacharjee, “Translation ranger: Operating system support for contiguity-aware tlbs,” in *Proceedings of the 46th International Symposium on Computer Architecture*, 2019, pp. 698–710.
- [54] A. Yoaz, M. Erez, R. Ronen, and S. Jourdan, “Speculation techniques for improving load related instruction scheduling,” in *Proceedings of the 26th annual International Symposium on Computer architecture*, 1999, pp. 42–53.
- [55] K. Zhao, K. Xue, Z. Wang, D. Schatzberg, L. Yang, A. Manousis, J. Weiner, R. Van Riel, B. Sharma, C. Tang, and D. Skarlatos, “Contiguitas: The pursuit of physical memory contiguity in datacenters,” in *Proceedings of the 50th Annual International Symposium on Computer Architecture*, 2023, pp. 1–15.